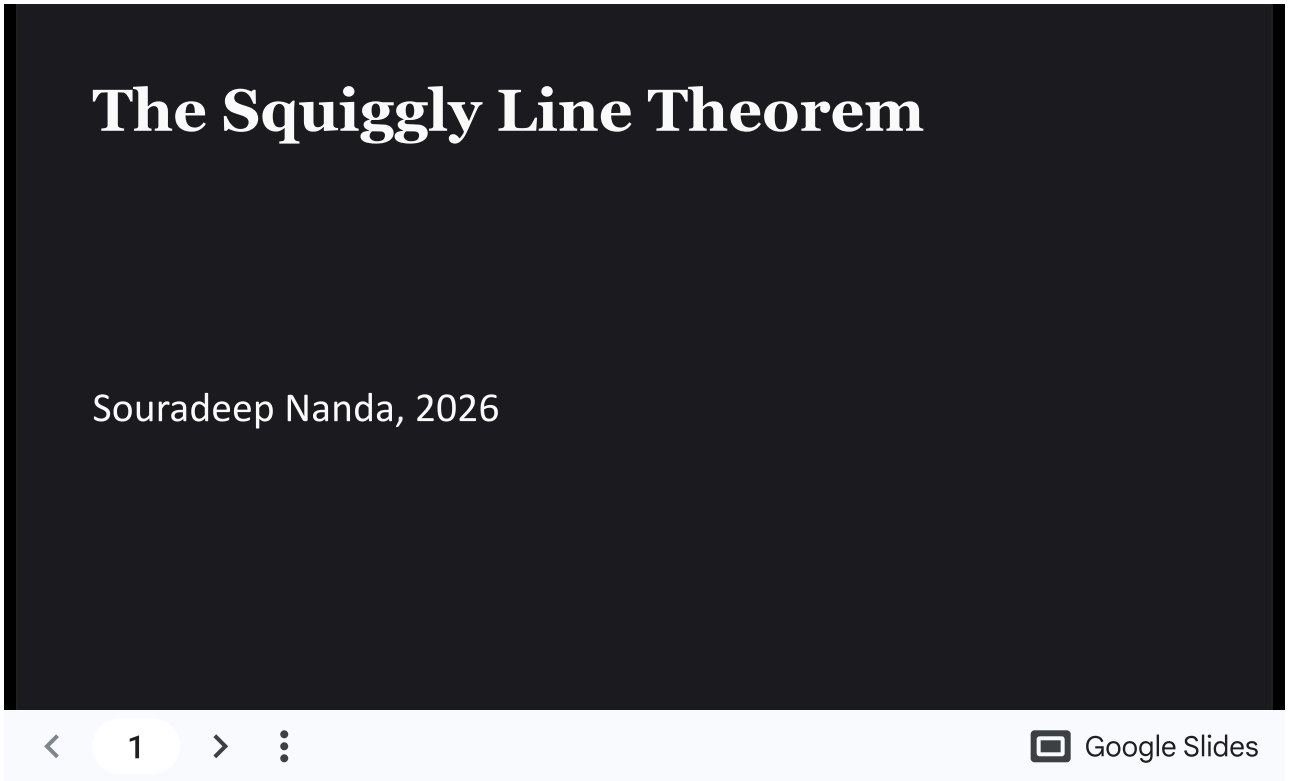
The Squiggly Line Theorem: An Idea From 2022 and What the Test Found

10 Nov 2026

A picture that was meant to tie together the lottery ticket hypothesis, double descent and faster learning in large networks. The geometry turned out to be true, and it is now a theorem for any squeezed function: closer, steeper, and no better at the start. A squigglier curve is closer to the data, in a direction that the loss at the start does not see. The learning claim did not hold.

Revised on 11 Nov 2026, after a review. The verdict is corrected: the learning claim is false, and the idea as I defined it is not established for networks. The tube derivation is replaced by an exact one, the initial-loss numbers are measured again with a script that is kept, the statements about length and about the wall are corrected, and there are new notes on a Lean formalisation and on a limit of the test. Revised a second time on 11 Nov 2026: the geometric claim is now proved for sines, and the post is renamed from "The Squiggly Line Conjecture". Revised a third time on 11 Nov 2026: the theorem no longer needs the sine, the first two claims are proved for a constructed ReLU family, and there is a new section on double descent.

In 2022, the year Stable Diffusion was released, I had an idea about why large networks learn fast. In 2025 I wrote the first scripts. In 2026 I tested it properly. There is also a slide version.



The short version: the idea is three claims in a chain. The first one holds, and it is now a theorem for any squeezed function. The other two fail as stated.

- (G) Geometry. A squigglier curve has a smaller nearest-point distance to random points. True, and proved for any squeezed function.
- (P) Parameters. More parameters give a squigglier random function. False for width with random weights; depth adds a little. True for a constructed ReLU family.
- (L) Learning. A smaller nearest-point distance means a smaller training loss at the start. False.

So a squigglier curve really is closer to random points, but in a direction that the loss at the start does not see. A shift of the input can use that nearness, at the price of a steep curve. The squiggly line theorem says it for any function: squeeze it sideways by a factor w, and it gets closer to the data, steeper, and no better at the start. A sine with a random phase is the standard example (see "What is proved in Lean"). I keep the name "squiggly line conjecture" for the idea of 2022: more parameters give a squigglier curve, which starts closer to the data and learns faster. As a statement about the initial loss it is false. For neural networks, the first two claims are proved for a constructed ReLU family, and what is still open is the claim for networks with random weights. There is one place where the idea is true: in parameter space, for models that are linear in their parameters, more parameters put a fit nearer to the random start, at an exchange rate of 1 / sqrt(c). That is the known lazy-training picture, not a new result. The link to the lottery ticket hypothesis was always a claim, and I did not derive it. For double descent I now have a partly proved account of one model (see "Double descent"). Below are the idea, my experiments, a correction to my derivation, and what published work says.

The original claim

The idea connects three puzzles of large networks in one geometric picture:

- The lottery ticket hypothesis: a large random network contains a small subnetwork that trains well.
- Double descent: test error falls, rises, then falls again as the model grows.
- Why larger networks learn faster.

I stated this link as a claim. I did not derive the three effects.

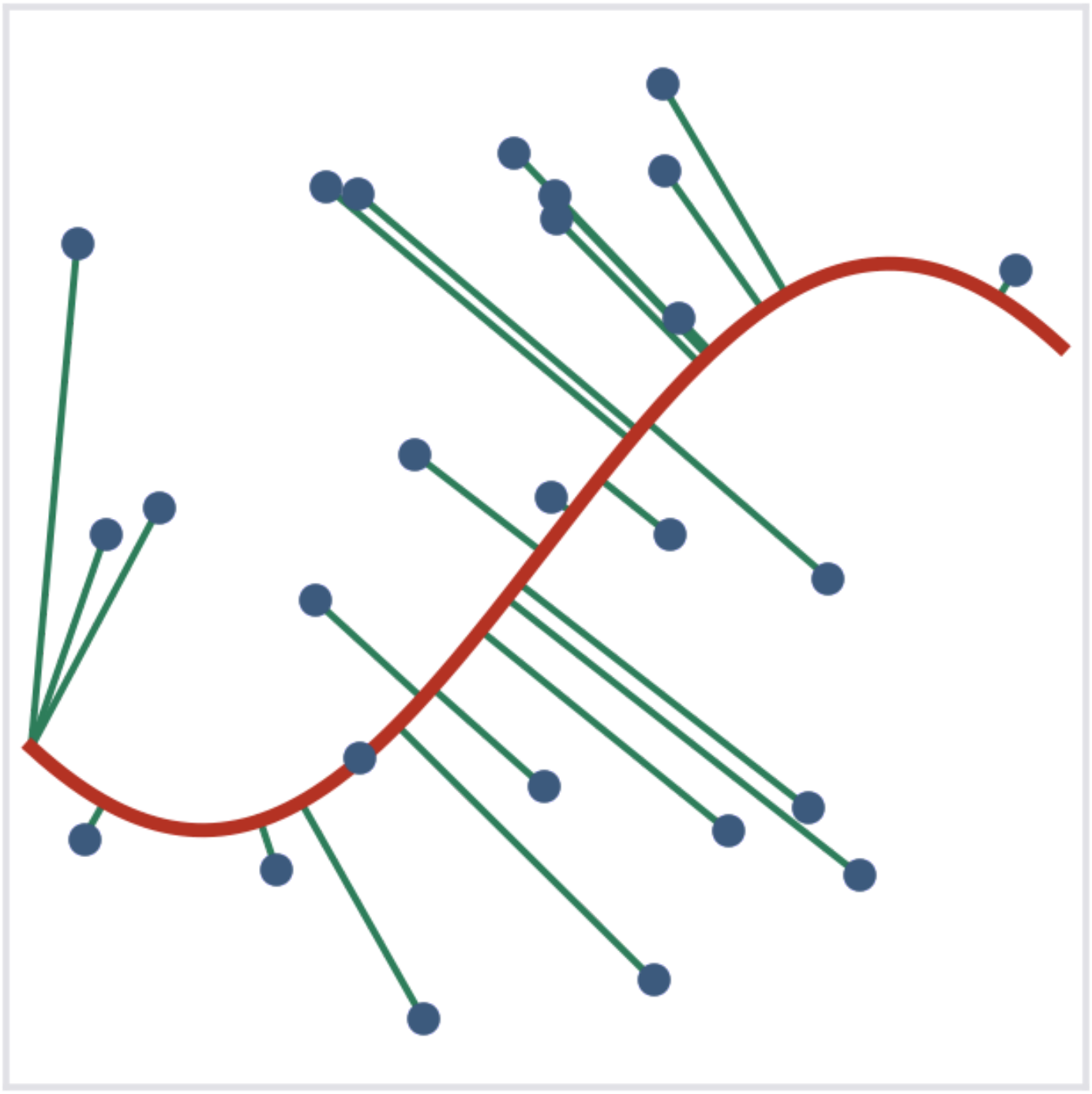
The core idea

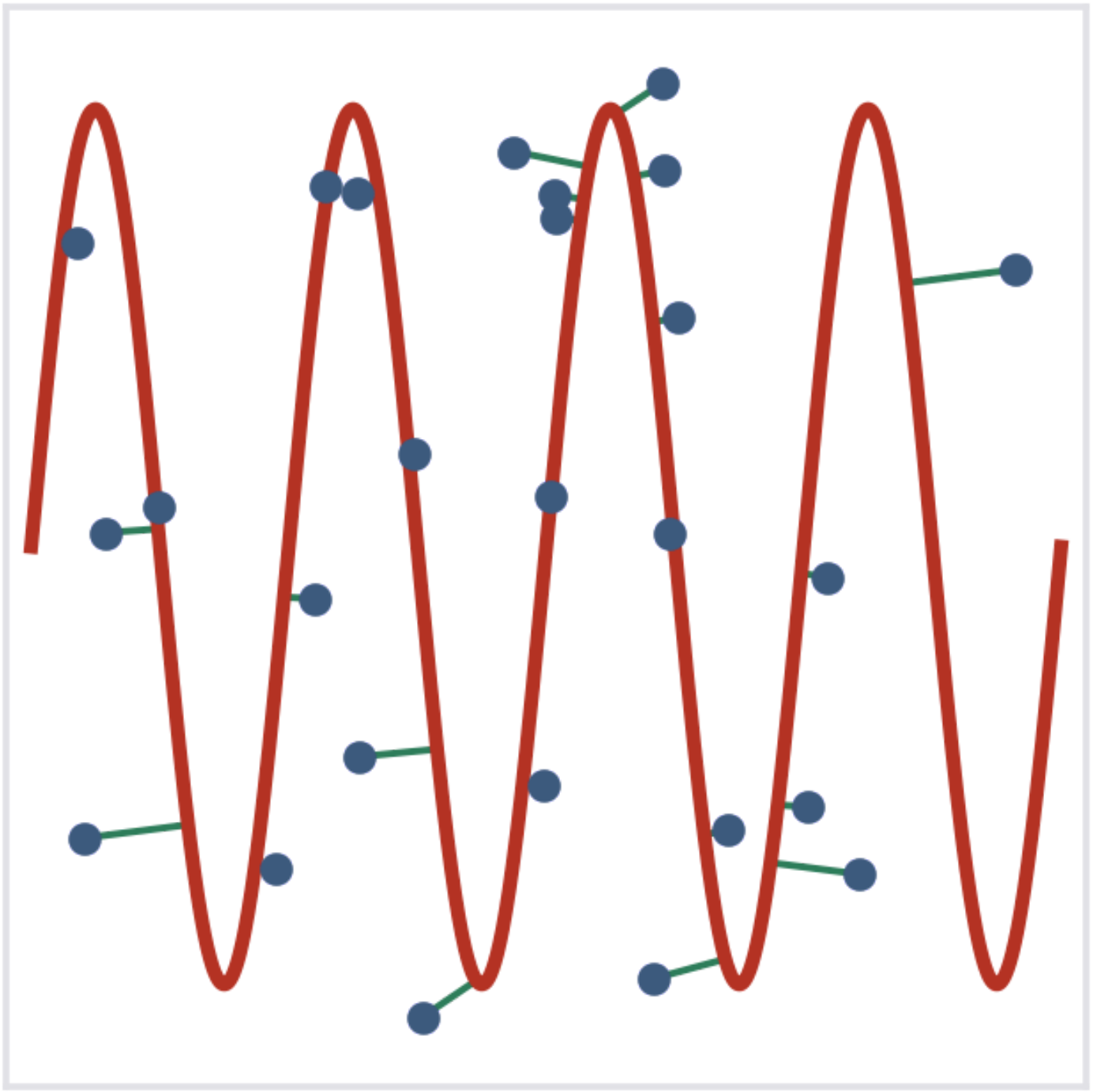
Take a scatter of data points and a curve (a "squiggly line") through the same space. The curve is the function that the network computes. Define the loss as the average distance from each data point to the nearest point on the curve:

```
L(f, D) = (1/n) * sum_i min_x || (x_i, y_i) - (x, f(x)) ||
data D in [-1, 1]^d x R
```

A "more squiggly" curve lies closer to the scatter. I measure squiggliness as the curve length:

```
l(f) = integral of sqrt(1 + ||grad f(x)||^2) dx
```

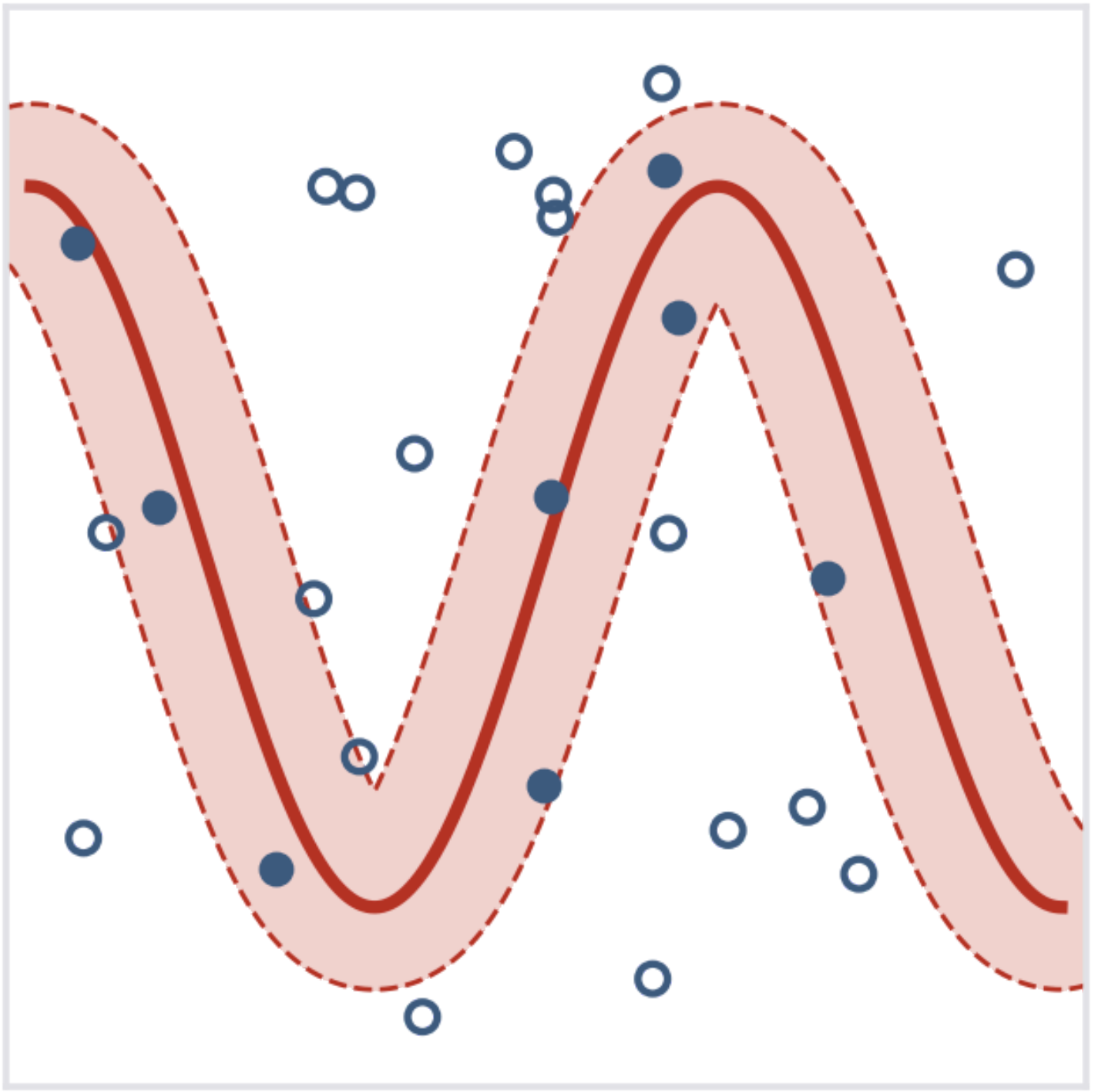




The conjecture: for two randomly initialised functions with parameter counts |theta_1| < |theta_2|, the expected loss E[L(f_theta_2, D)] is smaller than E[L(f_theta_1, D)]. More parameters, lower expected loss at initialisation.

The tube argument

Draw a tube of radius epsilon around the curve. The volume of the tube grows in proportion to the length: V_covered is about l * k * eps^(d-1). A random point is uncovered with probability V_uncovered / V_total. The original estimate was E[distance] about (V_uncovered / V_total)^(1/d).



At first I drew two conclusions from this. More length helps, but with diminishing returns. Perfect coverage needs infinite length: there is a wall, but it is at infinity. Also, a curve that folds or loops too tightly wastes length, because its tube overlaps itself.

ReLU networks

ReLU networks are piecewise linear, not polynomials. Where all units are active, the stacked layers collapse to one linear map. A dead unit makes the output constant. So the curve turns only at kinks, and squiggliness is a count of kinks.

Classification as a wind tunnel

In classification, each layer is a place where the wind can change direction. The data points are birds, and the wind carries them to a pigeonhole for each class. A deeper network is a longer tunnel with

finer control. In classification the squiggly line is not visible, because the output is a class.

Autoregression brings the line back

Autoregression is chained classification, so the line returns. The hidden state moves from token to token, and its path is a squiggly line through representation space. Attention lets each step read earlier points on the path.

A correction to the original derivation

In the 2026 review I found a sign error. My first derivation took E[dist] about (1 - l * k * eps^(d-1) / V)^(1/d) and concluded diminishing returns from a positive second derivative. For d greater than 1 the second derivative in l is negative (a negative exponent does not flip the sign). So that formula gives increasing returns, the opposite of my first conclusion.

Here is the exact form, the tube identity. The mean distance from a random point P to a set S is the probability that P is outside the tube of radius r around S, integrated over r:

```
E[dist(P, S)] = integral from r = 0 to infinity of Prob(P is outside the tube of radius r) dr
```

This replaces my first estimate, (uncovered fraction)^(1/d), which had no derivation. It also replaces the "slab" picture that the first version of this post used here.

For one input dimension the graph of f is a curve of length l. A tube of radius r around it has area at most about 2 l r. So in a region of area V the mean distance is at least about

```
E[dist] >= V / (4 l)
```

with equality only for a curve that is spread evenly. So V / (4 l) is a lower bound, not an estimate. I call it the tube bound below. Length inside the data region is necessary for a small distance. It is not sufficient.

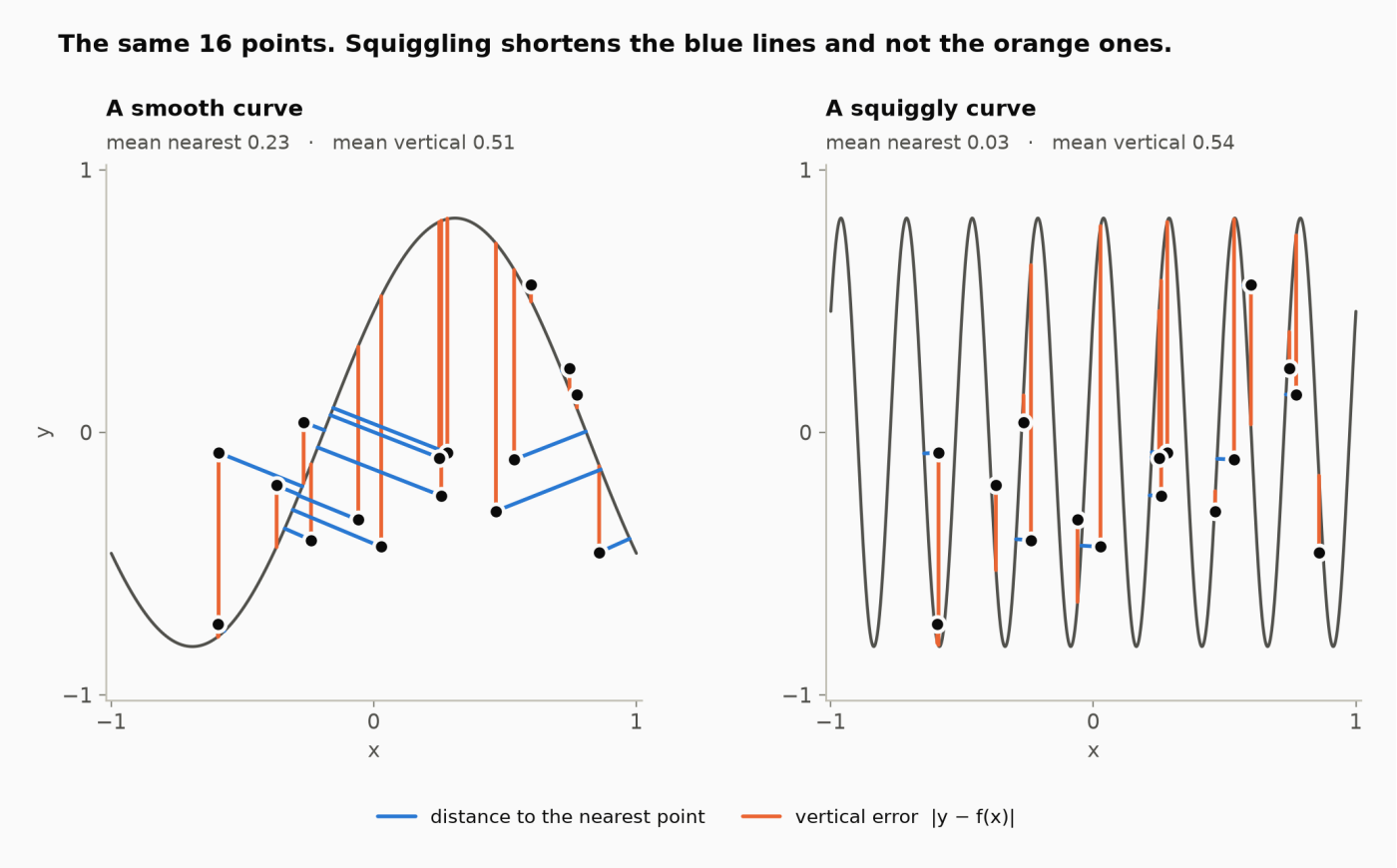
The bound is above zero at every finite length, so zero distance needs infinite length. For an evenly spread curve the distance equals the bound, which is convex in l, so the returns diminish. So the two conclusions stay, for a curve that can reach all the points.

For a 1-D path in D dimensions, for example a hidden-state trajectory, the same argument gives a bound of the order (V / (k l))^(1/(D-1)). This falls very slowly when D is large.

What my experiments found

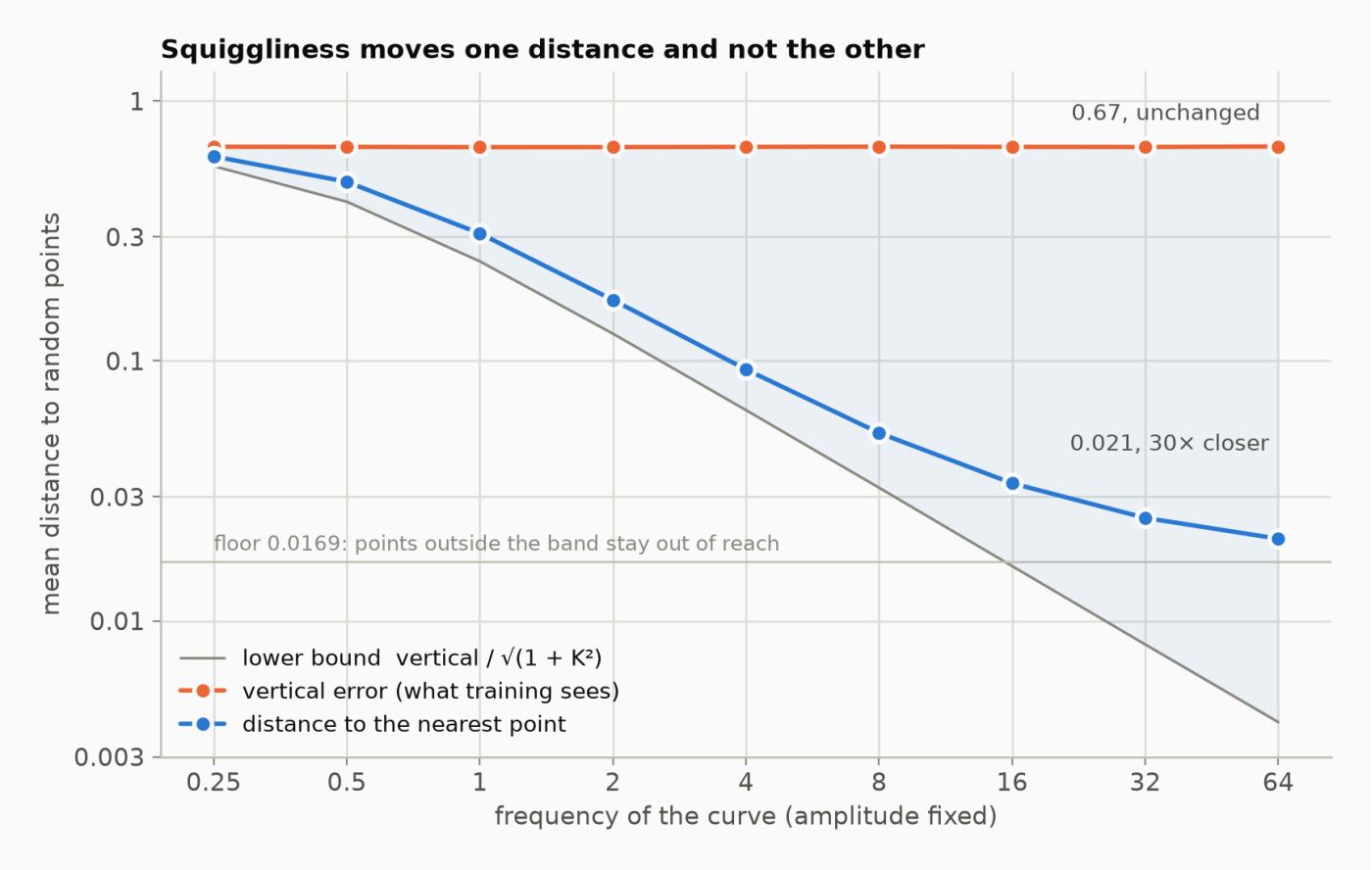
I ran these in September 2026. The first scripts from 2025 had looked like support for the idea, but their measurement code was wrong. One "distance" was really a count of active neurons. One scaling law was the network output fading to zero. So I started again with one clean experiment.

The experiment uses sine curves with one fixed amplitude and a random phase. Only the frequency changes, so at each x the curve has the same distribution of values for all frequencies. I measured two things against random points: the distance to the nearest point of the curve, and the vertical error |y - f(x)|.



This figure shows the two distances for one example: the same 16 points, a smooth curve and a squiggly curve. A blue line goes from a point to the nearest point of the curve. An orange line is the vertical error. On the squiggly curve the blue lines are much shorter (mean 0.03 against 0.23). The orange lines are not (mean 0.54 against 0.51).

Frequency	Curve length	Nearest-point distance	Vertical error
0.25	2.19	0.6129	0.6672
1	3.96	0.3095	0.6649
4	13.32	0.0928	0.6663
16	52.31	0.0338	0.6665
64	208.92	0.0207	0.6688



The figure is the same run as the table, with frequencies 0.5, 2, 8 and 32 added. The grey line is the lower bound of the slope sandwich, which I describe below.

The geometry holds. The nearest-point distance falls by a factor of 30.

The learning claim fails. The vertical error does not move. It stays between 0.6649 and 0.6688. A regression loss charges the vertical error. The wiggles let the curve pass near a point that is to the left or right of it, and the loss at the start gets nothing for that. The squiggly line is closer to the data in a direction that the loss does not measure at the start. A shift of the input can use it, as I describe under "Where it stands".

The flat column is an identity, not a finding. The expected loss of a random function depends only on the distribution of f(x) at each single x. How f moves between two inputs does not enter. A random phase gives the sine the same distribution at every x, for every frequency. So the column must be flat, and the spread from 0.6649 to 0.6688 is sampling noise. The identity is enough to refute the learning claim for these curves: the nearest-point distance falls by a factor of 30, and the expected loss cannot move.

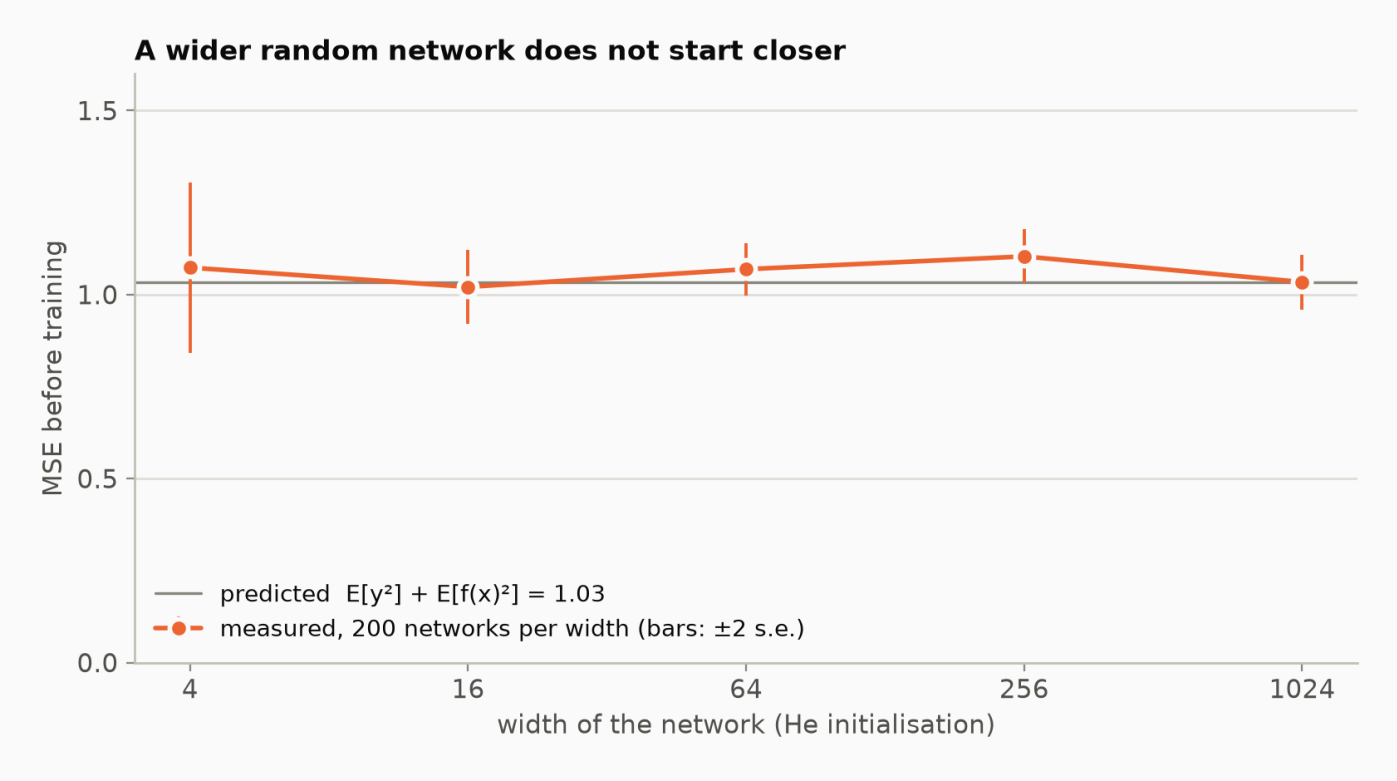
The slope sandwich. The two distances are tied together by the slope of the curve. If f is K-Lipschitz (its slope is never above K), then at every data point:

```
vertical / sqrt(1 + K^2) <= nearest <= vertical
```

So the nearest-point distance can be far below the vertical error only when the curve is steep. A small nearest-point distance means that a small change of the input reaches the label.

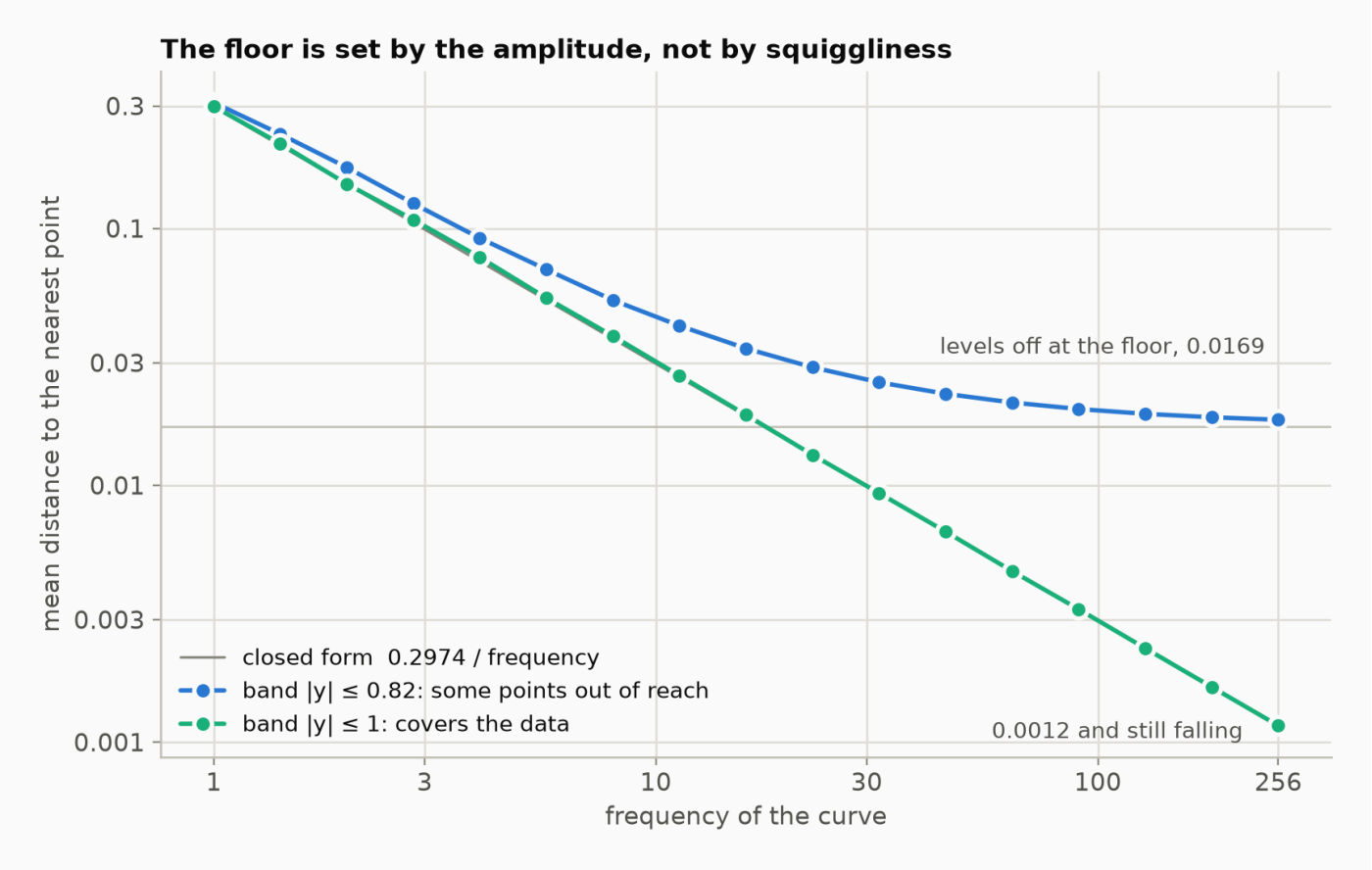
Length outside the data region is worth nothing. I stretched a straight line to the same length as the frequency-64 curve (209 units). Its distance was 0.50, against 0.02 for the squiggly curve. That is 24 times worse at the same length. But a straight line of length 209 over x in [-1, 1] has a slope of about 104, so about 99% of it lies above or below the data box. So this test shows that length outside the data region is worth nothing. It does not show that length is not the mechanism. By the tube bound above, length inside the region is required. What matters is that the length is inside the data region and spread across it.

Random networks agree. This is the same identity. Under He initialisation, E[f(x)^2] of a random ReLU network is the same at every width. So the expected mean squared error before training is E[y^2] + E[f(x)^2] at every width. I measured it with `scripts/initial_mse.py`: 10 inputs, 200 points, x and y uniform on [-1, 1], two hidden layers, 200 random networks per width. The measured values were 1.07, 1.02, 1.07, 1.10 and 1.03 for widths 4, 16, 64, 256 and 1024, against a predicted 1.03 at every width. It is flat. A wider network does not start closer. The first version of this post quoted other numbers here (2.97 to 3.13). The script that produced them was not kept, so I measured again with a stated setup.



The bars are 2 standard errors up and down. The grey line is the predicted value.

The floor comes from the amplitude, and the wall stays at infinity. At higher frequencies the distance stops falling at a floor. The floor comes from the amplitude: the curve can never reach points outside the band that it sweeps. For my amplitude the floor was 0.017. It does not depend on the number of points. With a band that covers all the data there is no floor. The distance keeps falling like 1/frequency: it tends to 0.2974 / frequency, where 0.2974 = (2 - 8/pi^2) / 4. So the wall is at infinity after all, as I first thought. The first version of this post had the heading "The wall is not at infinity" here. My own control contradicted it.



Blue is the amplitude of the experiment. Its band is from -0.82 to 0.82, and the distance levels off at the floor. Green is the control, with a band that covers the data. It follows 0.2974 / frequency and reaches 0.0012 at frequency 256.

What a second small check found

I made this check in November 2026, before I compared it with the experiments above. It has a 1-D input and no training. It measures only the nearest-point distance, so it tests the geometry and not the learning claim.

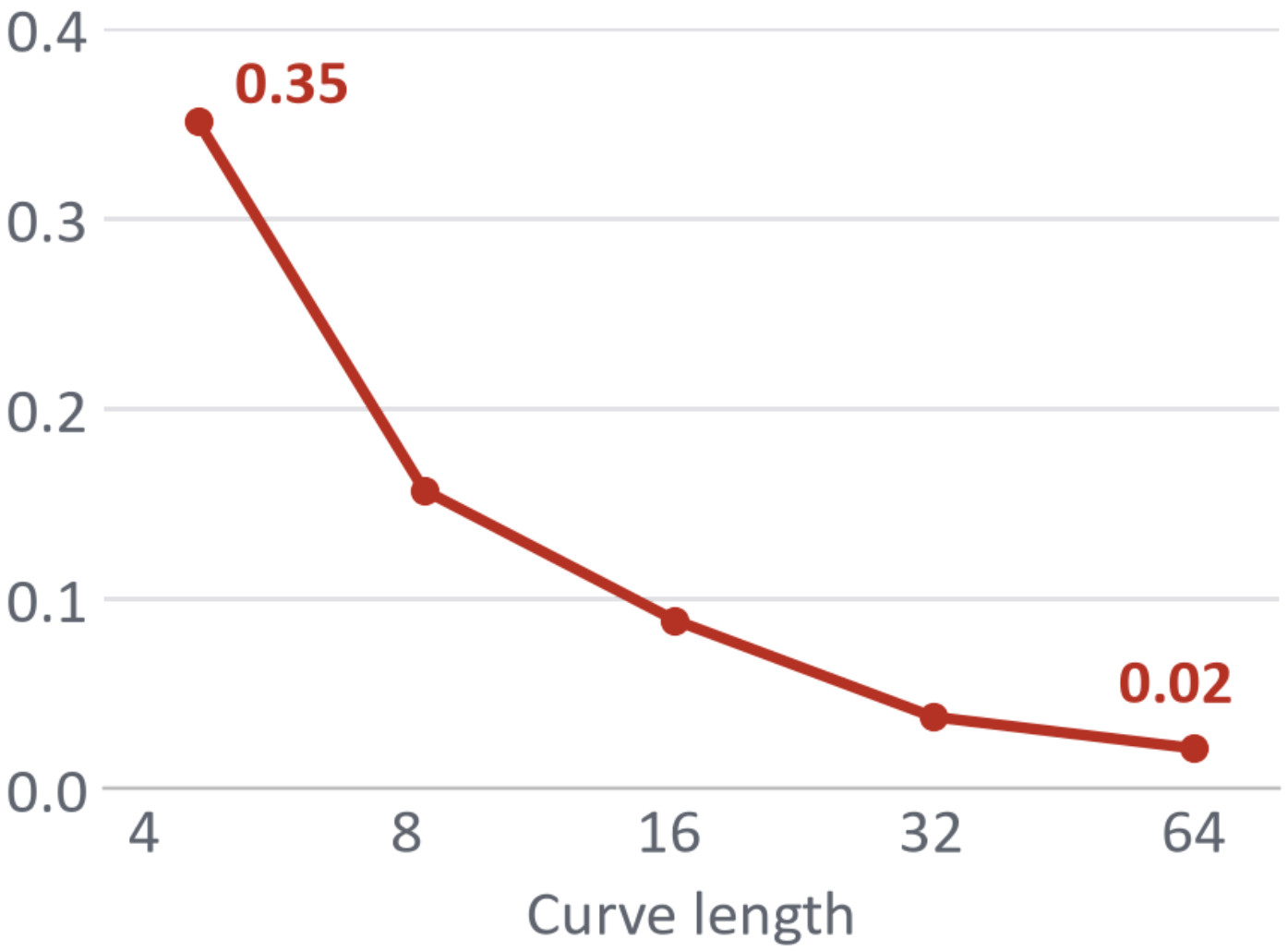
Reference curves

First I took sine curves with 1, 2, 4, 8 and 16 periods in a box of side 2, and measured the mean distance to 200 random points.

Periods	Length	Mean distance
1	4.6	0.35
2	8.4	0.16
4	16.2	0.09
8	32.1	0.04
16	64.1	0.02

Distance times length stays between 1.2 and 1.6. So the distance falls like 1/length. The geometry part of the idea holds.

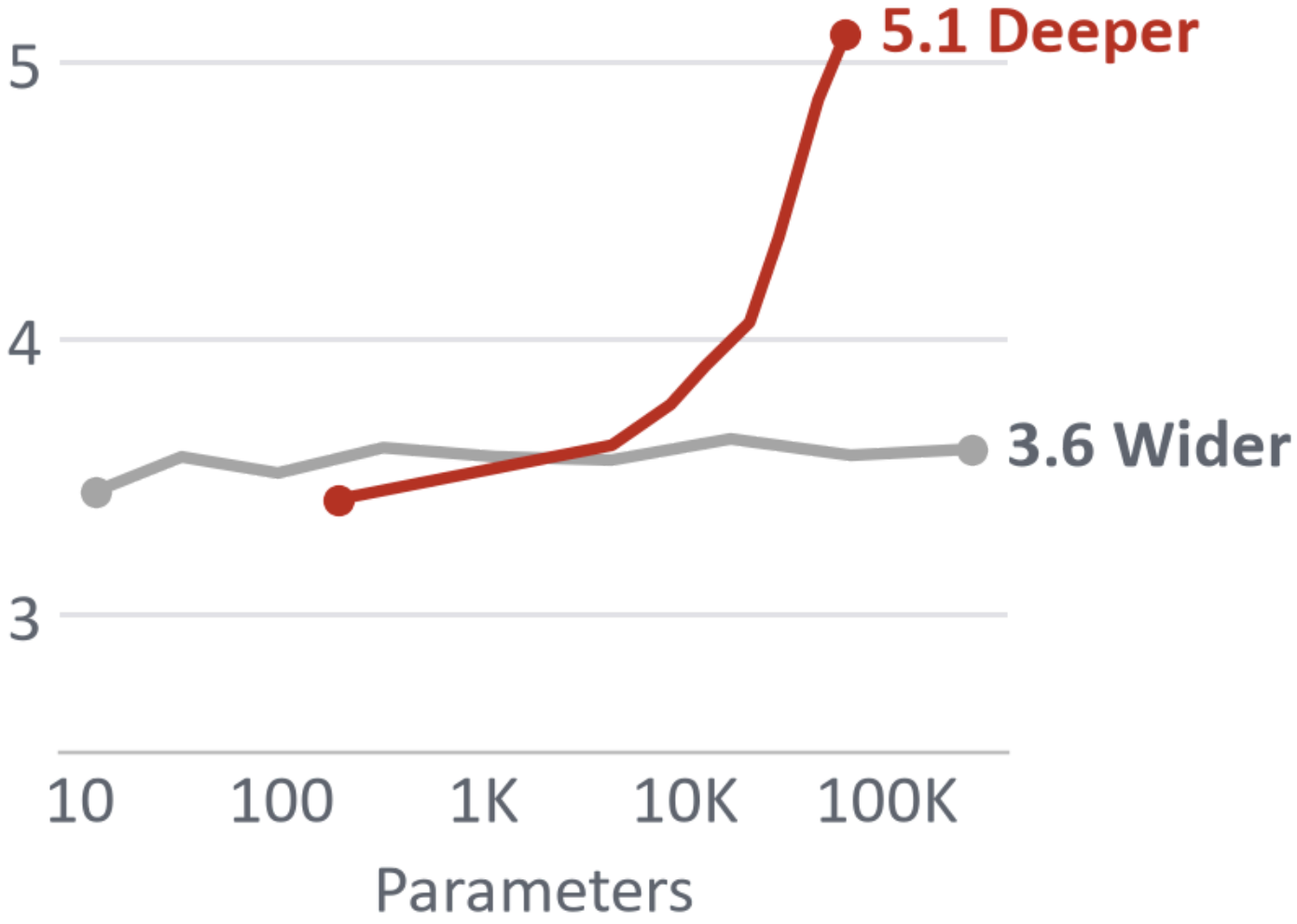
The tube bound above says that this product is at least 1.0, with equality only for an evenly spread curve. The measured values are 1.2 to 1.6, and there is a reason. For a sine that spans the box, distance times length tends to 2 - 8/pi^2 = 1.19 at high frequency, because the strokes of a sine are not evenly spaced at each height. I measured 1.186 at 256 periods. The larger values at low frequency are finite-frequency corrections.

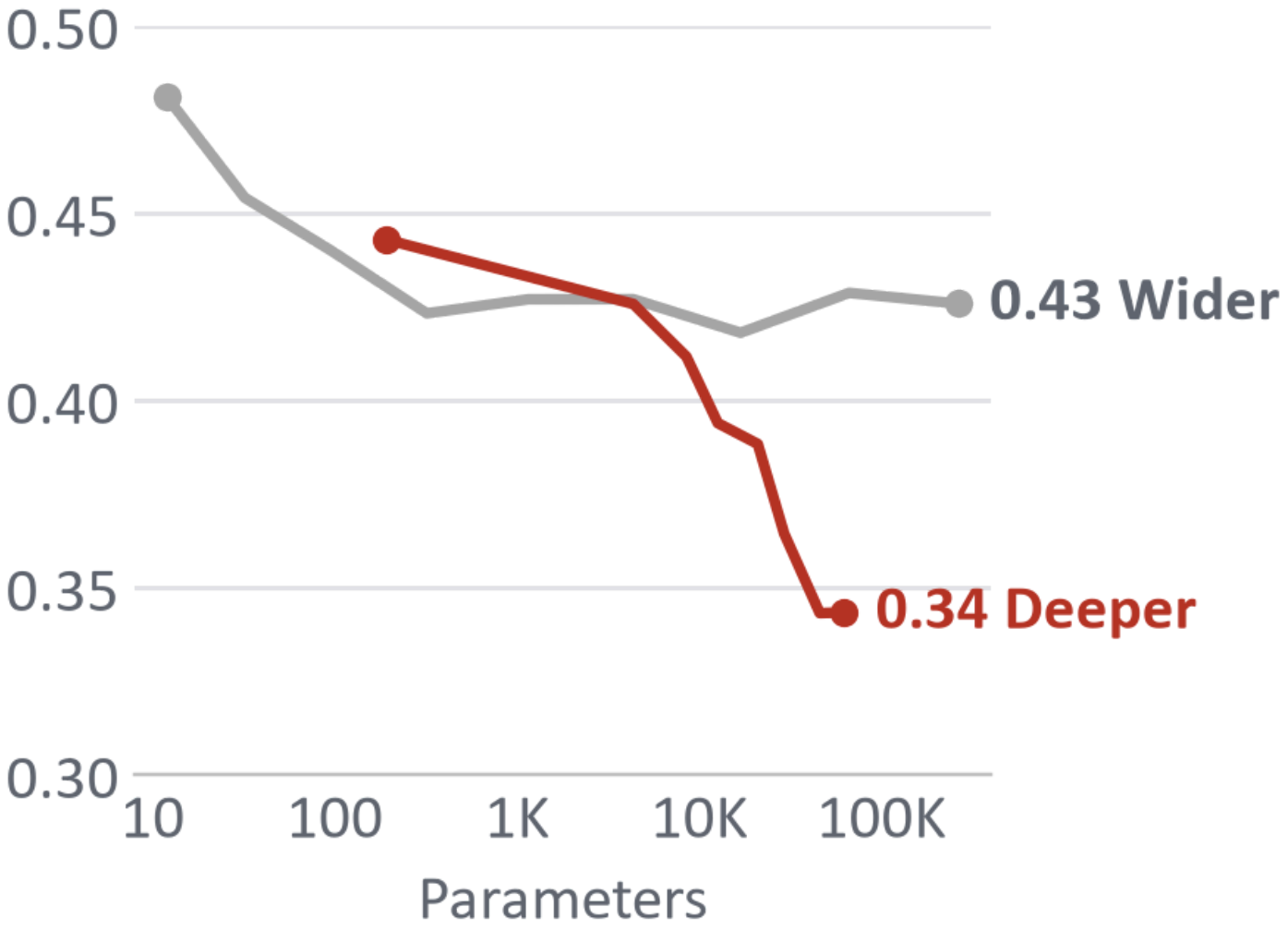


Random ReLU networks

Next I made 300 random ReLU networks of each size, with He-uniform weights. I rescaled the output to span the box.

- Wider networks (2 hidden layers, width 2 to 512): the curve length stays near 3.5 to 3.6 at all widths. The mean distance falls from 0.48 to about 0.42 and then stays flat from width 16.
- Deeper networks (width 64, 1 to 16 hidden layers): the length grows from 3.4 to 5.1. The distance falls from 0.44 to 0.34.
- Over 1,600 pooled random networks, the correlation between length and distance is -0.76.


5.1 Deeper
5
4
3.6 Wider
3
10
100
1K
10K
100K
Parameters



With the PyTorch default initialisation (smaller weights), the random curves are almost flat lines at every size. Their length is 2.0 to 2.1, where a flat line has length 2.0.

My reading: a longer curve is closer to the data in the nearest-point distance, but more parameters do not reliably give a longer curve. Width adds no length in this check. Depth adds some. This is claim (P) of the chain, and it fails for width with random weights. This agrees with the flat initial error in the experiments above.

Where the idea is true

The sine family has one random number, so it cannot test a claim about parameter count. `scripts/fourier_params.py` uses a family where the count is the only dial: a Fourier series with m terms and random coefficients, scaled so that the output has the same variance at every m. There are 20 points with random labels, and a fit of all points exists once there are 20 parameters.

Parameters	Initial MSE	Nearest-point distance	Travel to a fit	Gradient-descent steps
9	0.65	0.229	no fit	never
21	0.65	0.128	448	188 000 000
25	0.64	0.110	35.5	79 300

Parameters	Initial MSE	Nearest-point distance	Travel to a fit	Gradient-descent steps
65	0.71	0.056	1.61	166
129	0.66	0.027	0.48	13
513	0.63	–	0.23	3

"Travel" is the distance in parameter space from the random start to the nearest parameter vector that fits every point. "Steps" is plain gradient descent to a mean squared error of 0.001 with the largest stable step size.

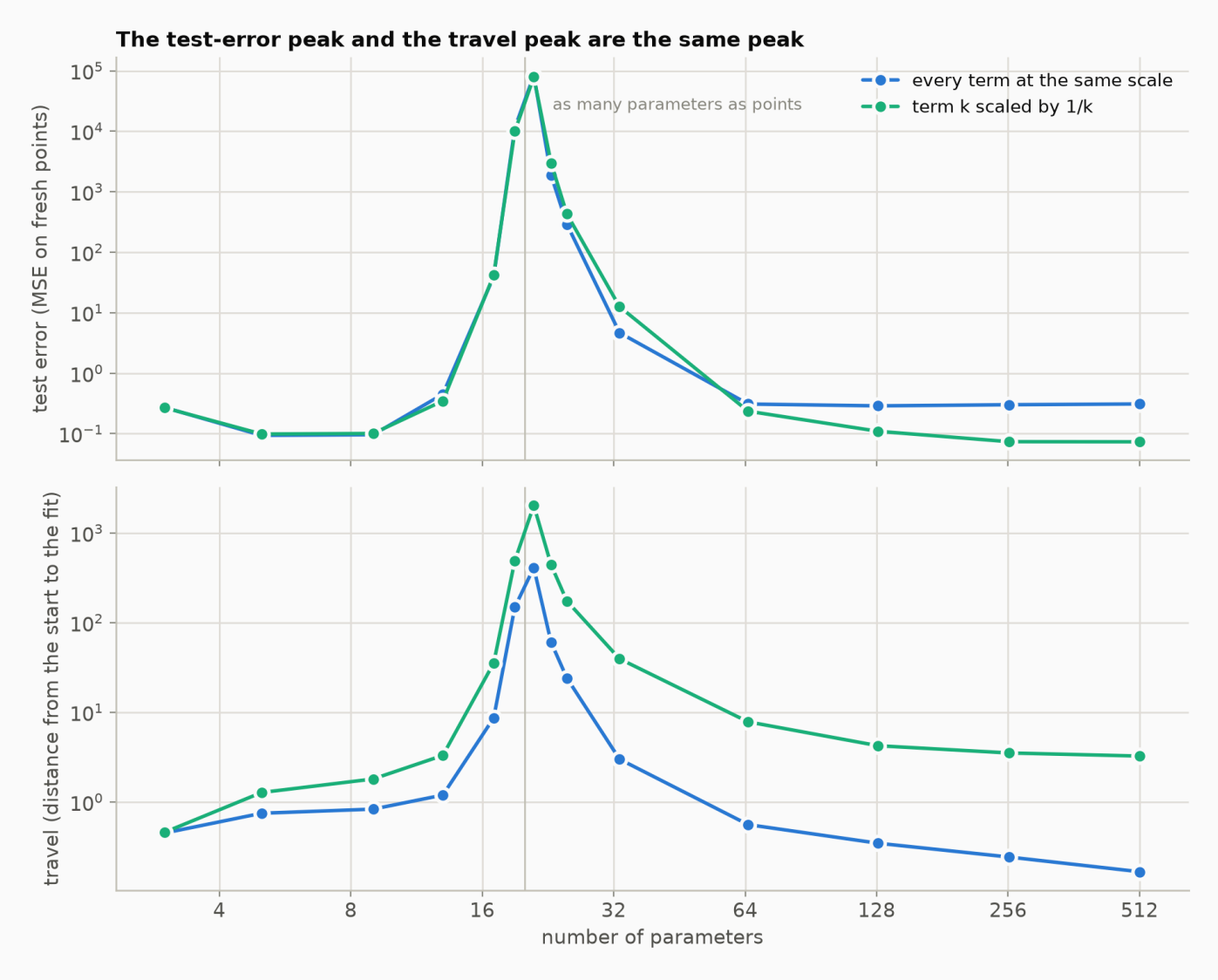
A larger random model does start closer to a trained one, and it trains in far fewer steps. But the closeness is in parameter space. The initial loss is the same at every size (0.63 to 0.71, sampling noise around 2/3), as the marginal lemma requires. The nearest-point distance also falls, like 1/parameters, because these random series get steeper as terms are added. The worst case is right at the threshold, 21 parameters for 20 points, which is where double descent has its peak.

Far above the threshold, the travel is about sqrt(n x initial MSE / (m + 1)) for n points and m terms. That gives 0.45, 0.32 and 0.22 for 129, 257 and 513 parameters, against 0.48, 0.34 and 0.23 measured. So the distance in parameter space is the root of the loss, divided by a factor that grows like the root of the parameter count. This is the form in which my original idea is true. It is the known lazy-training picture, not a new result.

Double descent

The original idea said that the picture ties in with double descent, and derived nothing. `scripts/double_descent.py` fits the same Fourier model to 20 noisy points and scores it on fresh ones.

Parameters	Test error	Travel to the fit
5	0.094	0.75
13	0.45	1.2
21	83 800	409
33	4.7	3.0
65	0.31	0.56
513	0.31	0.17



The test error falls to 0.094, peaks at 83 800 where the model has as many parameters as points, and falls again. The travel peaks at the same place. I proved two facts that connect them: a fit that is near the start gives a tame function everywhere, and the travel is at most the size of the labels divided by sqrt(c), where c is the weakest direction of the kernel. Along one direction of the labels the peak is exact. If the model cannot reach that direction, no fit exists and the model leaves it alone. If it reaches it with strength lambda, the travel is exactly its size divided by sqrt(lambda). So being unable to fit the noise is safe, being barely able to is the worst case, and being easily able to is safe again. That the test error itself peaks there is measured, not proved. How low the second descent goes depends on which terms are cheap: with every term at the same scale it ends at 0.31, no better than predicting zero (0.32), and with term k scaled by 1/k it ends at 0.073, below the first minimum.

What is proved in Lean

The research repository has a Lean 4 project on Mathlib. It states the conjecture and the learning claim and proves what follows. There is no `sorry` in the project, each theorem depends only on Lean's three standard axioms, and a GitHub workflow rebuilds the proofs on every change.

- **The squiggly line theorem for any function** (`squiggly_line_theorem_general`). Squeeze any function, with any number of inputs, sideways by a factor w. It is closer: if the function

reached every label within rho of every input, the squeezed one is within rho / w of every data point. It is steeper: its slope is w times larger, and its distance to a data point is at least `vertical / sqrt(1 + (w K)^2)`. It is no better at the start: if the function is random with the same output distribution at every input, the expected loss is unchanged, for every regression loss. And a small shift cashes it in: any target is fitted after moving each input by at most rho / w.

- **The sine, the standard example** (`squiggly_line_theorem`). For a random-phase sine A sin(w x + phi) and data whose labels lie inside the band the sine sweeps, at every data point and every phase, `vertical / sqrt(1 + (A w)^2) <= nearest <= pi / w`. The expected nearest-point distance tends to zero as the frequency grows, and the expected loss is the same at every frequency.
- **Its pieces.** The slope sandwich, the tube identity, the one-point marginals (a regression loss sees only the distribution of f(x) at each single x), and the covering bound (a function that reaches every label within rho of every input is within rho of the data).
- **The verdict.** The learning claim fails for random-phase sines, for every regression loss and every data distribution.
- **Parameter space, for models linear in their parameters.** A movement of size e buys the fraction e / d of the loss. Adding parameters never moves the nearest fit further away. When the features are orthogonal on the data with squared length c, the distance to the nearest fit is exactly the size of the residual divided by sqrt(c). Gradient descent follows the kernel, so two models with the same kernel have the same path and the same travel.
- **Double descent, for the same models.** A fit near the start is tame everywhere. The travel is at most the size of the residual divided by sqrt(c) when the kernel is at least c in every direction. Along a direction of strength lambda no fit exists if lambda is 0, and the travel is exactly the size of the residual divided by sqrt(lambda) if lambda is above 0.
- **ReLU networks.** L layers of two neurons pass within 1 / 2^L of every point of the unit square (`nearestDist_iterate_tent_le`). A network with one neuron per layer never turns, and L layers of two neurons make 2^L - 1 turns. L folds are one Fourier term of frequency 2^L. ReLU in the plane acts as a polarizer, and a chain of polarizers, which is a network of linear layers, delivers its target with an error of at most pi^2 / (8N) after N plates.

Not formalised: the exact constant 0.2974 for sines, the V / (4 l) bound, the rate at which travel falls with parameter count (about 1 / sqrt(parameters)), that the test error peaks at the threshold, and the statements about networks with random weights.

I have also written a paper, "The Squiggly Line Theorem: Closer, Steeper, and No Better at the Start", with these results. It is a draft and it has not been reviewed.

What published work says

The summary first: published work backs the geometry (length and coverage) and does not back the parameter claim. That matches my experiments. Here is each paper with my verdict.

Raghu et al., "On the Expressive Power of Deep Neural Networks" (ICML 2017). The length of the image of an input path grows exponentially with depth when the weights are large. Width matters

little. This is for the idea, for depth only.

Hanin, Jeong and Rolnick, "Deep ReLU Networks Preserve Expected Length" (ICLR 2022). At standard (He) initialisation the expected length does not grow with depth. It shrinks slightly. This is against the random-initialisation form of the conjecture. My straight-line test does not remove this objection: by the tube bound, length inside the data region is required.

Dherin et al., "The Geometric Occam's Razor Implicit in Deep Learning" (2021). It uses the same arc-length measure. SGD pushes the length down, and a trained 1-D ReLU network is near the shortest curve through the data. This is against "longer is better" after training.

Ramanujan et al., "What's Hidden in a Randomly Weighted Neural Network?" (CVPR 2020). A wider random network contains a better subnetwork. This is for the lottery-ticket link. But the gain is in the best subnetwork after a search, not in the average random function.

Larsen, Fort, Becker and Ganguli, "How many degrees of freedom do we need to train deep networks" (ICLR 2022). A random subspace of weight space reaches the low-loss set once it has enough dimensions. This is a coverage argument like mine, but in weight space, not in data space.

Bubeck and Sellke, "A Universal Law of Robustness via Isoperimetry" (NeurIPS 2021). To fit n noisy points smoothly, a network needs many parameters. More parameters allow a smoother fit. So parameters reduce the squiggliness that is necessary. They do not add it.

Chiang et al., "Loss Landscapes are All You Need" (ICLR 2023). Random sampling of weights until the training data are fit ("guess and check") generalises about as well as SGD, and wider networks do better. This supports "size helps without training". But the functions that fit are the simple ones.

Hosseini and Fedorenko (NeurIPS 2023). Trained language models make the token trajectory straighter, and larger models straighten more. This is against the squiggly reading of the token path.

Martinelli, Brea and Gerstner, "Lottery Tickets Do Not Explain Overparameterization. How About Escape Dimensions?" (2026 position paper). It argues that width helps by adding dimensions to escape bad minima, not by better luck at initialisation.

The 2026 revision

The first point is a result of my experiments and of the tube bound. The other two are proposals with no test.

1. Length inside the data region and spread across it, not parameter count. Length is necessary: the mean distance is at least about V / (4 l). Length is not sufficient: a straight line of the same length has about 99% of its length outside the data box, and it does 24 times worse.
2. Look at reachable curves, not one random curve. A larger network can reach more curves, by training or by a subnetwork search. Measure the coverage of that family.
3. Training spends length. Published results show shorter curves and straighter token paths after training. So squiggliness can be a starting resource, not the end state.

Possible errata

Here are the places where I think the idea or the check can be wrong.

1. The sign error described above.
2. The graph of f is a surface, not a curve, when the input has more than one dimension. The "length" integral is then a surface area, and the tube volume is area times 2 eps.
3. The original distance estimate, (uncovered fraction)^(1/d), is stated with no derivation. The tube identity above replaces it.
4. The loss is not the training loss. It is the distance to the nearest point of the curve. Regression trains on the vertical error, which is never smaller. For the sine curves the effect of squiggliness on the expected vertical error is zero, and this is an identity. This is the erratum that ends the learning claim.
5. The data are assumed to fill the box. Real data lie near a function. A curve that covers all the space also fits noise, so generalisation is not addressed.
6. The links to the lottery ticket hypothesis, double descent and faster learning are claims with no derivation.
7. The experiments and the check are 1-D (the initial-loss measurement has 10 inputs). They are my own scripts, not a published benchmark.
8. A numerical correction. The routine for the distance from a point to the curve picked the wrong stroke of the curve once the strokes were closer together than the sample spacing. This overestimated one control by 11% at frequency 1000. I fixed it on 11 Nov 2026. No number from the first version of this post changes: the frequency table is identical to every digit.
9. The test family is limited. A random-phase sine has one degree of freedom. It can never fit more than two points at once, at any frequency. The mean nearest-point distance scores each point alone, so it cannot see this. That is why the number of points dropped out of every result. The lottery ticket hypothesis and double descent are about fitting n points jointly.

Next steps

The original question is answered, so these are new questions. The full list, with a status for each, is in the open problems. Reachability is now addressed for models linear in their parameters (see "Where the idea is true"). For ReLU networks it would only repeat what the lazy-training work already says.

1. The lower tail. The mean initial error is flat in width. Is the chance of a low initial error also flat?
2. Subnetworks. Compare the best subnetwork of a random network with the full network, on the vertical error.
3. Training. Find out if a trained network keeps, removes or moves its kinks.
4. Joint fit (untested). Score a joint fit of all the points, not single points. Under a random-function prior with kernel K, the cost of fitting labels y is y^T K^-1 y. For random labels its mean is tr(K^-1). At fixed amplitude this is smallest when K is the identity, that is, for the least correlated (squiggliest) prior. For smooth targets the order reverses. This is a proposal. I have not tested it.

Where it stands

The first version of this post said here that the conjecture is true, and true in a direction that training cannot use. Both halves needed correcting. Of the three claims in the chain, only the first holds. And the nearness can be used, as the last paragraph of this section says.

- (G) A squigglier curve has a smaller nearest-point distance to random points. True, and proved for any squeezed function.
- (P) More parameters give a squigglier random function. False for width with random weights; depth adds a little. True for a constructed ReLU family.
- (L) A smaller nearest-point distance means a smaller training loss at the start. False.

The accurate summary is four statements. There is a theorem for any squeezed function, and the random-phase sine is its standard example. The idea is true in parameter space, for models that are linear in their parameters: more parameters put a fit nearer to the random start, at an exchange rate of 1 / sqrt(c). It is false as a statement about the initial loss, because the vertical error, which training minimises, does not change with squiggliness, and a wider random network does not start with a lower loss. And for neural networks the first two claims are proved for a constructed ReLU family: L layers of two neurons draw a curve that passes within 1 / 2^L of every point of the unit square. A network with one neuron per layer never turns, and a chain of polarizers, which is a network of linear layers, delivers its target with an error of at most pi^2 / (8N) after N plates. What is still open is the claim for networks with random weights: how the oscillation scale of a random network depends on width, depth and weight scale. So the picture does not explain why large networks learn fast.

The nearness is not useless, though. It can be cashed in by a small shift of the input: if the curve is within rho of a data point, shifting the input by at most rho makes the curve pass through that point. For one point this is a shift of the curve, which for a sine is a change of phase. For many points it needs a different small shift at each point, which is a small warp of the input, and that is what an earlier layer can do. This is proved: a curve that reaches every label within rho of every input fits any target after a warp that moves no input by more than rho. It is not shown that training finds the warp. The price is slope: a fit bought this way is steep and fragile.

I am not claiming priority for any of this. The open problems are listed in the repository.

Slides: The Squiggly Line Theorem

What do you think?

0 Responses

 Upvote  Funny  Love  Surprised  Angry  Sad

0 Comments 1 Login ▾

Start the discussion…

LOG IN WITH OR SIGN UP WITH DISQUS

Name

♡ • Share Best Newest Oldest

All articles

Causal-Consistency Training: An Idea From 2021 and the Research Since
10 Nov 2026

InSQuaD: Picking In-Context Examples That Are Relevant and Diverse
10 Nov 2026

The Squiggly Line Theorem: An Idea From 2022 and What the Test Found
10 Nov 2026

Appeasing the Machine Spirit: RLHF in In-Context Learning
03 Nov 2026

Four Tricks at the Edge of Formal Language Theory
15 Aug 2026

Spring-Loaded Dinosaurs: What Penguins, Kiwis, and Armored Core Get Right (and Wrong) About Legs
28 Feb 2026

Passive Velocity Sorting: A Geometric Maxwell's Demon?
14 Dec 2025

AC Gravity and Gravitational Opacity
05 Dec 2025

Theorem: Equivalence of -arccoth and tan Functions
21 Nov 2025

The Photonic Transformer Hypothesis: Rethinking Photosynthesis and Methane Consumption in Plants
19 Nov 2025

The Scalable Quasi-Perpetual Photonic Machine
17 Nov 2025

How to Make a Ball Orbit Itself
08 Nov 2025

RETRACTED: The Boron-Nitrogen Catastrophe: A Critical Gap in Beta Decay Verification
28 Oct 2025

X-Ray Data Pipeline: Ultra-High-Speed Communication Through Limestone Tubes
26 Oct 2025

The Universal Approximation Theorem Is Right. You're Using It Wrong.
19 Oct 2025

The Power Law Illusion: A Measurement Artifact Hypothesis
11 Oct 2025

Fractional Gamma Function via Fractional Derivatives
18 Sep 2025

On the cruel irony of the P-NP problem
18 Dec 2023

The Journey from India to Germany: A Guide for IT Professionals
20 Jun 2023

How does AutoGPT work under the hood?
03 May 2023

Unit Test Recorder - Automatically generate unit tests as you use your application
25 Jun 2020

How to migrate from vanilla Kubernetes to Istio service mesh?
14 Oct 2019